

Non-Generic Queue Collection Class in C# with Examples

In this article, I am going to discuss the **Non-Generic Queue Collection Class in C#** with Examples. Please read our previous article where we discussed the [Non-Generic Stack Collection Class in C#](#) with Examples. The Non-Generic Queue Collection Class in C# represents a First-In, First-Out collection of objects. That means we need to use this collection when we need First-In, First-Out access to items of a collection. The Queue Collection Class belongs to the System.Collections namespace. At the end of this article, you will understand the following pointers.

1. [What is a Queue in C#?](#)
2. [Methods, Properties, and Constructor of Non-Generic Queue Collection Class in C#](#)
3. [How to Create a Queue in C#?](#)
4. [How to Add Elements into a Queue in C#?](#)
5. [How to Remove Elements from the Queue in C#?](#)
6. [How to get the first element of the queue in C#?](#)
7. [How to check whether an element exists or not in the Queue in C#?](#)
8. [How to Clone the Non-Generic Queue Collection in C#?](#)
9. [How to Copy a Queue to an Existing Array in C#?](#)

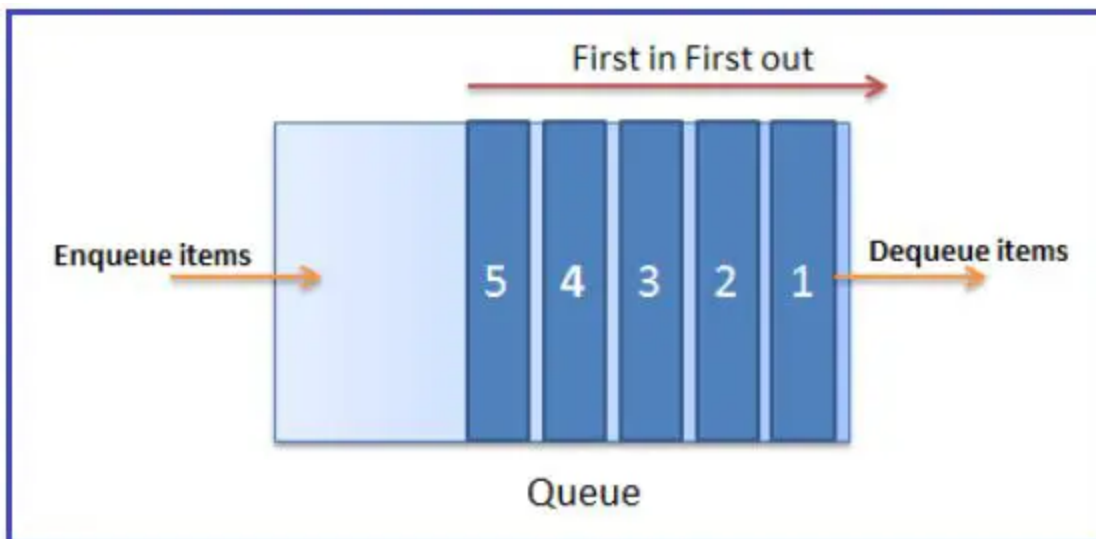
What is a Queue and How Does the Queue Collection Work in C#?

The Non-Generic Queue Collection Class in C# works in the FIFO(First-In-First-Out) principle. So, we need to use the Non-Generic Queue Collection class in C#, when we need First-In-First-Out access to the items of a collection. That means the item which is added first will be removed first from the collection. When we add an item to the queue, then it is called enqueueing an item. Similarly, when we remove an item from the queue then it is called dequeuing an item.

In order to understand Queue collection, first, we need to understand the FIFO Principle as Queue works on the FIFO Principle. Let us understand the FIFO principle with an example. Imagine a queue of people waiting for a ticket in a cinema hall. Normally, the first person who enters the queue will be the first person to get the ticket from the counter. Similarly, the last person who enters the queue will be the last person to get the ticket from the counter. For a better understanding, please have a look at the following image.



In C#, the Non-Generic Queue Collection also works in the same way. Elements are added to the queue in a sequential manner. When we add an item to the queue, then it is called Enqueueing an item. The process of adding an element to the Queue is called Enqueue operation. Similarly, when we remove an item from the queue then it is called Dequeueing an item. This operation is known as Dequeue. For a better understanding, please have a look at the below image.



Note: Queue is defined as both Generic and Non-Generic types of Collections in C#. The **Generic Queue** Collection is defined in **System.Collections.Generic** namespace whereas Non-Generic Queue Collection is defined under **System.Collections** namespace. Here in this article, we will be discussing the Non-Generic Queue Collection Class in C# with Examples.

Characteristics of Non-Generic Queue Collection Class in C#:

1. The Enqueue method adds an element to the end of the Queue.
2. The Dequeue method removes the oldest element from the start of the Queue.

3. The Peek method returns the oldest element that is at the start of the Queue but does not remove it from the Queue.
4. The capacity of a Queue is the number of elements the queue can hold. As we add elements to a queue, the capacity of the queue is automatically increased.
5. The Non-Generic Queue Collection allows both null and duplicate values.

Methods, Properties, and Constructor of Non-Generic Queue Collection Class in C#:

If you go to the definition of Queue Collection Class in C#, then you will see the following signature. Here, you can see that the Non-Generic Queue Collection class implements the IEnumerable, ICollection, and ICloneable interfaces.

```
namespace System.Collections
{
    public class Queue : ICollection, IEnumerable, ICloneable
    {
        public Queue();
        public Queue(ICollection col);
        public Queue(int capacity);
        public Queue(int capacity, float growFactor);

        public virtual int Count { get; }
        public virtual bool IsSynchronized { get; }
        public virtual object SyncRoot { get; }

        public static Queue Synchronized(Queue queue);
        public virtual void Clear();
        public virtual object Clone();
        public virtual bool Contains(object obj);
        public virtual void CopyTo(Array array, int index);
        public virtual object Dequeue();
        public virtual void Enqueue(object obj);
        public virtual IEnumerator GetEnumerator();
        public virtual object Peek();
        public virtual object[] ToArray();
        public virtual void TrimToSize();
    }
}
```

How to create a Queue in C#?

The non-generic Queue collection class in C# has provided four constructors which we can use to create a queue. The constructors are as follows:

1. **Queue():** It is used to initialize a new instance of the Queue class that is empty and has the default initial capacity, and uses the default growth factor.

2. **Queue(ICollection col):** It is used to initialize a new instance of the Queue class that contains elements copied from the specified collection and has the same initial capacity as the number of elements copied and uses the default growth factor. Here, the parameters col specifies the System.Collections.ICollection to copy elements from.
3. **Queue(int capacity):** It is used to initialize a new instance of the Queue class that is empty, has the specified initial capacity, and uses the default growth factor. Here, the parameter capacity specifies the initial number of elements that the Queue can contain.
4. **Queue(int capacity, float growFactor):** It is used to initialize a new instance of the Queue class that is empty, has the specified initial capacity, and uses the specified growth factor. Here, the parameter capacity specifies the initial number of elements that the Queue can contain and the parameter growFactor specifies the factor by which the capacity of the Queue is expanded.

Let's see how to create a queue using the Queue() constructor in C#:

Step1:

As the Queue collection class belongs to **System.Collections** namespace, so first, we need to include System.Collections namespace in our program with the help of the "using" keyword as follows:

```
using System.Collections;
```

Step2:

Next, we need to create an instance of the Queue class using the Queue() constructor as follows:

```
Queue queue = new Queue();
```

How to Add Elements into a Queue in C#?

If we want to add elements to a queue, then we need to use the Enqueue() method of the Queue class.

1. **Enqueue(object obj):** This method is used to add an object to the end of the Queue. Here, the parameter obj specifies the object to add to the Queue. The value can be null

Example to Understand How to Create a Queue and Add Elements in C#:

For a better understanding of how to create a queue and how to add elements to a queue in C#, please have a look at the below example.

```
using System;
using System.Collections;
namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
            queue.Enqueue(67.8);
        }
    }
}
```

```

        queue.Enqueue('A');
        //Printing the queue items using foreach loop
        foreach (var item in queue)
        {
            Console.WriteLine(item);
        }

        Console.ReadKey();
    }
}

```

Output:

```

101
Hello
3.14
True
67.8
A

```

How to Remove Elements from the Queue Collection in C#?

In Queue, you are allowed to remove elements from the beginning of the Queue. If you want to remove elements from the queue, then you need to use the following two methods provided by the Non-Generic Collection Queue class.

1. **Dequeue():** This method is used to remove and return the object at the beginning of the Queue. It returns the object that is removed from the beginning of the Queue.
2. **Clear():** This method is used to remove all objects from the queue.

Let us see an example to understand the Dequeue() and Clear() methods of the Queue Collection Class in C#. Please have a look at the below example which shows the use of the Dequeue and Clear methods.

```

using System;
using System.Collections;
namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
        }
    }
}

```

```

queue.Enqueue(67.8);
queue.Enqueue('A');
//Printing the queue items using foreach loop
Console.WriteLine($"All Queue Elements: Count {queue.Count}");
foreach (var item in queue)
{
    Console.Write($"{item} ");
}
//Removing and Returning an item from the queue using the Dequeue
Console.WriteLine($"
Deleted Element: {queue.Dequeue()}");
//Printing item after removing the first added item
Console.WriteLine($"
All Queue Elements After Deletion: Count
{queue.Count}");
foreach (var item in queue)
{
    Console.Write($"{item} ");
}
//Printing Items After Clearing the Queue
queue.Clear();
Console.WriteLine($"
Queue Elements After Clear Operation: Cour
{queue.Count}");
foreach (var item in queue)
{
    Console.Write($"{item} ");
}
Console.ReadKey();
    }
}
}

```

Output:

```

All Queue Elements: Count 6
101 Hello 3.14 True 67.8 A

Deleted Element: 101

All Queue Elements After Deletion: Count 5
Hello 3.14 True 67.8 A

Queue Elements After Clear Operation: Count 0

```

How to get the First Element of the Queue Collection in C#?

The Non-Generic Queue Collection class in C# provides the following two methods to get the first element of the queue collection

1. **Dequeue():** The Dequeue() method of the Queue class is used to Remove and return the object from the beginning of the Queue. If there is no object (or element) present in the Queue and if we are trying

to remove an item or object from the Queue using the pop() method then it will throw an exception i.e. **System.InvalidOperationException**

2. **Peek():** The peek() method of the Queue class is used to return the oldest object i.e. the object present at the start of the Queue without removing it. If there is no object (or element) present in the Queue and if we are trying to return an item (object) from the Queue using the peek() method then it will throw an exception i.e. **System.InvalidOperationException**

For a better understanding, please have a look at the below example which shows how to get the first element from the queue using both Dequeue and Peek Methods.

```
using System;
using System.Collections;
namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating a Queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
            queue.Enqueue(67.8);
            queue.Enqueue('A');
            Console.WriteLine($"Total Elements present in Queue : {queue.Count}");
            // Fetch First Element of Queue Using Dequeue method
            Console.WriteLine($"First Element of Queue is {queue.Dequeue()}");
            Console.WriteLine($"Total Elements present in Queue : {queue.Count}");
            // Fetch the topmost element from Queue Using Peek method
            Console.WriteLine($"First Element of Queue is {queue.Peek()}");
            Console.WriteLine($"Total Elements present in Queue : {queue.Count}");
            Console.ReadKey();
        }
    }
}
```

Output:

```
Total Elements present in Queue : 6
First Element of Queue is 101
Total Elements present in Queue : 5
First Element of Queue is Hello
Total Elements present in Queue : 5
```

Note: If you want to remove and return the first element from the queue, then use the Dequeue method and if you only want to return the first element from the queue without removing it, then use the Peek method and this is the only difference between these two methods of Queue Collection Class in C#.

How to Check Whether an Element Exists or not in the Queue Collection in C#?

If you want to check whether an element exists or not in the queue collection, then you need to use the following Contains() method of the Queue Collection Class in C#. You can also use this Contains() method to search for an element in the given queue.

1. **Contains(object obj):** This method is used to determine whether an element is in the Queue. Here, the parameter obj specifies the object or element to locate in the Queue. The value can be null. It returns true if obj is found in the queue; otherwise, false.

Let us understand this with an example. The following example shows how to use the Contains() method of the Queue class in C#.

```
using System;
using System.Collections;
namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating a Queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
            queue.Enqueue(67.8);
            queue.Enqueue('A');
            Console.WriteLine("All Elements of Queue");
            foreach (var item in queue)
            {
                Console.Write($"{item} ");
            }
            Console.WriteLine("\nChecking if the element Hello is present in 1
not");
            // Checking if the element Hello is present in the Stack or not
            if (queue.Contains("Hello"))
            {
                Console.WriteLine("Element Hello is found");
            }
            else
            {
                Console.WriteLine("Element Hello is not found");
            }
            if (queue.Contains("World"))
            {
                Console.WriteLine("Element World is found");
            }
        }
    }
}
```

```

        else
        {
            Console.WriteLine("Element World is not found");
        }
        Console.ReadKey();
    }
}
}

```

Output:

```

All Elements of Queue
101 Hello 3.14 True 67.8 A
Checking if the element Hello is present in the queue or not
Element Hello is found
Element World is not found

```

Note: The Contains(object obj) method of the Non-Generic Collection Queue Class in C# takes O(n) time to check if the element exists in the queue. This should be taken into consideration while using this method.

How to Clone the Non-Generic Queue Collection in C#?

If you want to clone the Non-Generic Queue collection in C#, then you need to use the following Clone() method provided by the Queue Collection Class.

1. **Clone()**: This method is used to create and return a shallow copy of a Queue object.

For a better understanding, please have a look at the below example.

```

using System;
using System.Collections;
namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
            queue.Enqueue(67.8);
            //Printing All Queue Elements using For Each Loop
            Console.WriteLine("Queue Elements:");
            foreach (var item in queue)
            {

```

```

        Console.WriteLine(item);
    }
    //Creating a clone queue using Clone method
    Queue cloneQueue = (Queue)queue.Clone();
    Console.WriteLine("\nCloned Queue Elements:");
    foreach (var item in cloneQueue)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
}
}

```

Output:

```

Queue Elements:
101
Hello
3.14
True
67.8

Cloned Queue Elements:
101
Hello
3.14
True
67.8

```

How to copy a queue to an existing array in C#?

In order to copy a queue to an existing array in C#, we need to use the following CopyTo method of the Non-Generic Queue Collection Class.

1. **CopyTo(Array array, int index):** The CopyTo method of the Non-Generic Queue Collection Class in C# is used to copy the System.Collections.Queue elements to an existing one-dimensional System.Array, starting at the specified array index. Here, the parameter array specifies the one-dimensional Array that is the destination of the elements copied from Queue. The Array must have zero-based indexing. The index parameter specifies the zero-based index in the array at which copying begins. If the parameter array is null, then it will throw ArgumentNullException. If the parameter index is less than zero, then it will throw ArgumentOutOfRangeException.

This method works on one-dimensional arrays and does not change the state of the queue. The elements are ordered in the array in the same way as the order of the elements from the start of the queue to the end. Let us see an example for a better understanding.

```

using System;
using System.Collections;

```

```

namespace QueueCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a queue collection
            Queue queue = new Queue();
            //Adding item to the queue using the Enqueue method
            queue.Enqueue(101);
            queue.Enqueue("Hello");
            queue.Enqueue(3.14f);
            queue.Enqueue(true);
            queue.Enqueue(67.8);
            //Printing All Queue Elements using For Each Loop
            Console.WriteLine("Queue Elements:");
            foreach (var item in queue)
            {
                Console.WriteLine(item);
            }
            //Copying the queue to an object array
            object[] queueCopy = new object[5];
            queue.CopyTo(queueCopy, 0);
            Console.WriteLine("\nQueue Copy Array Elements:");
            foreach (var item in queueCopy)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Queue Elements:
101
Hello
3.14
True
67.8

Queue Copy Array Elements:
101
Hello
3.14
True
67.8

```

Properties of Queue Class in C#

1. **Count**: It gets the number of elements contained in the Queue.
2. **IsSynchronized**: It gets a value indicating whether access to the Queue is synchronized (thread-safe). It returns true if access to the queue is synchronized (thread-safe); otherwise, false. The default is false.
3. **SyncRoot**: It gets an object that can be used to synchronize access to the Queue. It returns an object that can be used to synchronize access to the queue.

Non-Generic Queue Collection Class in C# Overview

The following are some important points that you need to remember while working with Queue Collection.

1. In C#, Queues are used to store a collection of objects in a FIFO (First in, First out) style, i.e., the element which is added first will remove first.
2. By using Enqueue() method, we can add elements at the end of the queue.
3. The Dequeue() method will remove and return the first element from the queue.
4. The queue Peek() method will always return the first element of the queue, and it won't delete elements from the queue.

In the next article, I am going to discuss the [Non-Generic SortedList Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Non-Generic Queue Collection Class in C#** with Examples. I hope this Non-Generic Queue Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.